

20 — Lo scorrimento perduto: chi possiede la posizione della pagina

Un cuore da accendere riportava in cima alla collezione. Nessuno scriveva `scrollTo`: la pagina si accorciava per un decimo di secondo, e il browser faceva il resto. Cos'è il *clamping* dello scorrimento, perché ridisegnare tutto è una decisione e non un dettaglio, e come si conferma un'interfaccia ottimista senza il ridisegno che prima rimetteva a posto le bugie. Il guasto era uno, i gesti che lo facevano tre — cuore, copie, «la voglio»: le altre due sorelle stanno nella sezione 7, con le trappole dello stato condiviso che si vedono solo alla seconda. Esempi: `src/ui/griglia-collezione/griglia-collezione.js`, `src/app/app.js`.

1. Il guasto

Nella collezione ogni card ha un cuore in alto a sinistra: un tocco e la carta entra nei preferiti. Scorrendo la collezione se ne accendono dieci, quindici, una dopo l'altra — è proprio il modo in cui la funzione si usa.

Il difetto, riportato da chi usava l'app: **a ogni cuore la pagina tornava in cima**. Per mettere dieci carte nei preferiti bisognava risalire la collezione dieci volte.

Il codice sospetto è lungo tre righe, in `app.js`:

```
g.addEventListener('preferita-cambiata', async (evento) => {
  const { idSet, numero, preferita } = evento.detail;
  await impostaPreferita(idSet, numero, preferita);
  await aggiornaCollezione(); // ← qui
});
```

`aggiornaCollezione()` rilegge tutta la collezione dal database e la riassegna alle griglie. E il *setter* della griglia fa una cosa sola:

```
set voci(valore) {
  this.#voci = valore ?? [];
  this.#disegna(); // → this.innerHTML = `...`
}
```

Cioè: **per accendere un cuore si ricostruiva l'intera pagina**. Nessuna riga tocca lo scorrimento. Eppure lo scorrimento si perdeva.

2. Chi possiede la posizione della pagina

La posizione di scorrimento **non è uno stato del documento**: è uno stato del browser, che vive accanto al documento e deve restare valido rispetto ad esso. La regola è banale a dirsi e ha conseguenze grosse:

Non si può stare scrollati oltre la fine del contenuto.

Quando il layout viene ricalcolato, il browser *taglia* (in inglese **clamp**) l'offset di scorrimento nell'intervallo ancora possibile, cioè fra 0 e `scrollHeight - clientHeight`. Se il contenuto si accorcia mentre sei in fondo, non ti resta la posizione: ti resta il massimo consentito da ciò che è rimasto.

Ecco la misura, presa nel browser prima della correzione. La pagina mostrava le carte trovate con una ricerca, l'utente era a 8000px, e ha toccato un cuore:

ms dal tocco	scrollHeight	scrollTop	card in pagina
0	12277	8000	63
50	1960	1148	3
358	12277	8000	63

La riga di mezzo è il guasto intero. `innerHTML` ha buttato via sessanta card, la pagina è diventata alta 1960px, e 8000 non era più una posizione legale: il browser l'ha tagliata a 1148. Da lì in poi il danno è fatto — il contenuto che ritorna non riporta indietro l'utente.

Qui il contenuto tornava dopo 358ms perché quella ricerca si rifà da sola. Nel caso peggiore — le carte mancanti, che arrivano set per set mentre scorri — non torna affatto: quelle card ricompaiono solo se qualcuno ripassa da lì scorrendo, e l'utente nel frattempo è in cima.

E lo *scroll anchoring*?

I browser hanno una difesa automatica: lo **scroll anchoring**. Il browser sceglie un elemento visibile come ancora e, se sopra di lui cambia l'altezza, corregge l'offset per tenerlo fermo sotto i tuoi occhi. È quello che ti salva quando una pubblicità carica sopra l'articolo che stai leggendo.

Non poteva salvare noi, ed è utile capire perché: **l'ancora è un nodo**. Se distruggi tutto il sottoalbero, l'ancora non esiste più e non c'è niente da tenere fermo. Lo scroll anchoring compensa i cambi *attorno* al contenuto che guardi; non sa ricostruire un contenuto che hai sostituito in blocco.

La lezione generale: le difese automatiche del browser presuppongono che il DOM sia *modificato*. Se lo **rifai**, il browser non ha modo di sapere che è "lo stesso" contenuto di prima.

3. Il ridisegno totale è una decisione

Ricostruire tutto a ogni cambiamento è un modo legittimo di scrivere una UI: è il modello dichiarativo, e per un pannello piccolo è la scelta più semplice e più difficile da sbagliare. Ma è una decisione con un prezzo, e il prezzo si paga in proporzione a cosa c'è a schermo.

Chi viene da Angular ha già visto il conto da vicino, sotto altri nomi: `*ngFor` senza `trackBy` distrugge e ricrea ogni riga a ogni giro, e con essa perde stato del DOM, focus, posizione. `trackBy` esiste esattamente per dire al framework «questa riga è la stessa di prima». Un framework può offrirtelo perché tiene una rappresentazione dell'albero da confrontare. **Qui non c'è nessun framework: la parte di `trackBy` la scrivi tu, o non c'è.**

Cosa si perdeva davvero, a ogni cuore:

cosa	perché tornava caro
le carte mancanti caricate scorrendo	ricaricabili solo ripassando col dito su quel set
le immagini già in pagina	ripartono da <code>data-src</code> , l'osservatore le richiede
lo scorrimento	tagliato, come sopra
una rilettura completa del database	per un campo booleano di una riga

E l'informazione cambiata era una: `preferita: true` su una carta.

4. La cura: toccare solo la card che è cambiata

Il metodo nuovo, in `griglia-collezione.js`, è tutto qui:

```
aggiornaPreferita(idSet, numero, preferita) {  
  const uguale = (v) => v.idSet === idSet && String(v.numero) === String(numero);  
  const voce = this.#voci.find(uguale);  
  if (voce) voce.preferita = preferita;  
  
  if (this.#effettivi().preferito === 'solo') {  
    this.#disegnaRisultati();  
  }  
}
```

```

    return;
  }

  const cuore = [...this.querySelectorAll('[data-preferita]')].find(
    (c) => c.dataset.set === idSet && c.dataset.numero === String(numero),
  );
  if (cuore) this.#accendiCuore(cuore, preferita);
}

```

Tre cose meritano un commento.

Prima si aggiorna il dato, poi il DOM. #voci è la verità del componente: se non la aggiorni, il prossimo ridisegno per un altro motivo — un filtro, una quotazione — rimetterebbe il cuore com’era. È lo stesso motivo per cui in Java non aggiorni la JLabel senza aggiornare il campo del modello.

Il ridisegno non sparisce: si restringe al caso che lo esige. La vista Preferiti è la stessa griglia con {preferito: 'solo'} fisso: là un cuore spento non cambia l’aspetto di una card, la fa *sparire dall’elenco*. Quel caso va ridisegnato, e per fortuna è quello che nessuno sta guardando — il cuore si tocca nel catalogo, non nella vista che filtra i cuori.

Le voci sono oggetti condivisi. Le due griglie ricevono lo stesso array da aggiornaCollezione(), quindi la stessa voce è *lo stesso oggetto*. Scriverci una volta basta per il dato; ogni griglia deve però sistemarsi il proprio DOM. Di qui la forma in app.js:

```

const stato = await impostaPreferita(idSet, numero, preferita);
for (const altra of griglie) altra.aggiornaPreferita(idSet, numero, stato);

```

5. L’interfaccia ottimista, senza rete di sicurezza

Il cuore si accende **prima** che il database risponda: un interruttore che aspetta non sembra un interruttore. È la tecnica nota come *optimistic UI*, e ha sempre due metà:

1. mostrare subito l’effetto sperato;
2. **conciliare** con quello che è successo davvero.

Nella versione vecchia la seconda metà non era scritta da nessuna parte: la faceva per caso il ridisegno, che ripartendo dai dati del database ridisegnava comunque lo stato vero. Tolto il ridisegno, la conciliazione va scritta — ed è il motivo per cui impostaPreferita() **restituisce lo stato finale**:

```

export async function impostaPreferita(idSet, numero, preferita = true) {
  const riga = await leggi(STORE_COLLEZIONE, chiave(idSet, numero));
  if (!riga || riga.desiderata) return false; // + il livello dati sa dire di no
  ...
  return Boolean(preferita);
}

```

Su una carta della lista desideri il cuore non si mette: è una carta che non possiedi. Prima quel “no” arrivava a video per un effetto collaterale; adesso arriva come valore di ritorno, e la griglia lo applica. Il rimedio è più corto del guasto e dice a voce alta una cosa che prima era implicita: **chi ha l’ultima parola sullo stato è il database, non il tocco**.

Il dettaglio che il ridisegno nascondeva

C’era una seconda bugia coperta dal ridisegno. Il cuore è un SVG, e il suo riempimento era scritto nel markup:

```
`<path d="M12 20.4 ..." fill="${acceso ? 'currentColor' : 'none'}" ... />`
```

Il tocco ottimista cambiava la classe .acceso, non l’attributo fill: per un istante il cuore era rosso ma **vuoto**, e a riempirlo arrivava il ridisegno. Senza ridisegno restava vuoto per sempre. La correzione sposta l’aspetto dove sta tutto il resto dell’aspetto, cioè nel CSS:

```
.carta-griglia .cuore.accesso svg path {
  fill: currentColor;
}
```

Funziona per una regola della cascata che vale la pena sapere a memoria:

origine del valore	forza
attributo di presentazione (fill="none", width="20") qualunque selettore CSS, anche * in un foglio d'autore stile inline (style="...")	come una regola a specificità zero vince vince su entrambi

Un attributo di presentazione è un *default*, non una decisione: qualsiasi regola CSS lo scavalca. Il che dà anche il criterio di progetto — **lo stato sta in una classe, l'aspetto sta nel foglio**: un solo interruttore da girare, e nessun pezzo di aspetto che aspetta di essere riscritto in JavaScript.

6. Come si misura una cosa del genere

Il difetto durava 300ms: guardandolo a occhio si vede solo “la pagina è saltata”. Serve un campionamento. Questo si incolla nella console e non richiede altro:

```
const campioni = [];
const t0 = performance.now();
elemento.click(); // l'azione sospetta
for (let i = 0; i < 40; i++) {
  campioni.push([Math.round(performance.now() - t0),
    document.body.scrollHeight, window.scrollY]);
  await new Promise((r) => setTimeout(r, 50));
}
console.table(campioni);
```

Due avvertenze imparate sul campo:

- leggere `scrollHeight` **forza un ricalcolo del layout**. È esattamente ciò che serve qui (vogliamo il valore vero, istante per istante), ma è la ragione per cui la stessa lettura dentro un gestore di `scroll` è un difetto di prestazioni — vedi 16;
- un service worker serve i file dalla **sua** cache. Una prova “prima e dopo” che non passa da `caches.delete()` e da una nuova registrazione misura due volte lo stesso codice. È successo mentre si scriveva questa correzione.

Dopo la correzione, la stessa misura sulla stessa azione: `scrollHeight` fermo a 12277, `scrollY` fermo a 8000, e zero sostituzioni di figli sulla griglia (controllabile con un `MutationObserver` su `{childList: true}`).

7. Le tre sorelle

Il cuore era il primo di tre. Gli altri due sono arrivati subito dopo, segnalati dalla stessa persona nella stessa frase: «quando aggiungo ai preferiti o le voglio o aggiungo, lo scroll torna all'inizio». Sono tre gesti che si fanno **scorrendo**, cioè nel punto della collezione più lontano dalla cima:

gesto	metodo	cosa cambia davvero in pagina
cuore	aggiornaPreferita()	la classe del cuore, il bordo della card
+ / -	aggiornaQuantita()	il badge $\times N$, la testata del set, il riepilogo
□ «la voglio»	aggiornaDesiderio()	la card intera (da mancante a desiderio), il riepilogo

Tre metodi e non uno generico, perché le tre domande sono diverse. Ma tutti e tre hanno dovuto rispondere alla stessa domanda difficile, e vale la pena isolarla.

Non indovinare se la card deve restare: chiedilo al filtro

Un aggiornamento chirurgico funziona finché la modifica cambia **l'aspetto** di una card. Quando ne cambia *l'appartenenza all'elenco* il ridisegno serve davvero: l'ultima copia tolta con «solo ciò che ho» attivo, un desiderio comprato mentre guardi la lista dei desideri. La tentazione è enumerare quei casi in una catena di `if`. Non regge: dipende da sette filtri, e domani da otto.

La forma che regge è chiedere al filtro stesso, che è già l'unica autorità in materia (`raggruppa.js`, funzione pura, testata):

```
if (filtra([voce], this.#effettivi()).length !== Number(Boolean(card))) {
  this.#disegnaRisultati(); // "dovrebbe starci" e "ci sta" non coincidono
  return true;
}
```

Due fatti osservabili — *deve stare a schermo?* e *ci sta?* — confrontati fra loro. Non «com'era prima»: le voci sono **oggetti condivisi** fra le due griglie, quindi la prima che passa le ha già cambiate sotto il naso alla seconda, e “prima” a quel punto non esiste più per nessuno.

Due trappole dello stato condiviso

Il push che allunga l'array di qualcun altro. `aggiornaCollezione()` passa lo stesso array a tutte le griglie. Un desiderio nuovo va aggiunto alle voci che la griglia conosce, ma con `push` comparirebbe anche nelle altre viste senza che nessuno gliel'abbia detto:

```
this.#voci = [...this.#voci, voce]; // riferimento nuovo: è cosa di questa griglia
```

È la regola dell'immutabilità applicata per un motivo concreto e non per disciplina — la stessa che in Angular ti fa restituire un array nuovo invece di mutarne uno con `OnPush`.

La voce amputata. Su ogni card viaggia `card._voce`, e conteneva sette campi scelti a mano: quelli che serviva al visore. Bastavano finché la card era di sola lettura. Da quando una card *mancante* può diventare un desiderio sul posto, quella voce entra fra le voci della griglia — e una voce senza *serie* finisce nel gruppo sbagliato al primo cambio di filtro. L'elenco dei campi da copiare era diventato “tutti”, e la correzione è dirlo:

```
card._voce = { ...voce, mancante };
```

Morale trasferibile: **una copia parziale è un contratto**, e vale finché nessuno usa quell'oggetto per uno scopo nuovo. Quando lo scopo cambia, la copia parziale non dà errore — dà un dato che sembra giusto.

Dove la card resta, e perché non si sposta

Una carta appena desiderata cambia posto: da “fra le mancanti, in fondo alla sezione” a “fra le carte tue”. `aggiornaDesiderio()` **non** la sposta: la riscrive dov'è, col bordo del desiderio e il badge 1. Ci andrà al prossimo giro lungo.

Non è pigrizia, è la stessa scelta di tutta questa correzione: spostarla adesso vorrebbe dire toglierla da sotto il dito che l'ha appena toccata. Una UI "perfettamente coerente" a ogni istante, se per esserlo deve muovere ciò che stai guardando, è meno usabile di una che aspetta il momento buono.

Esercizi

1. **Il terzo caso che manca.** `aggiornaQuantita()` con `quantita === 0` ridisegna: la carta esce dall'elenco e la pagina si accorcia davvero. Progetta l'alternativa chirurgica (togliere la card e riscrivere i contatori) e di' quale problema *nuovo* introdurrebbe con le sezioni di set — cosa resta a schermo se quella era l'unica carta del set?
2. **Il caso che resta.** Nella vista Preferiti, togliendo un cuore, la card sparisce e la pagina si accorcia davvero. È lo stesso difetto? Motiva la risposta guardando *quanto* si accorcia e *dove* si trova l'utente.
3. **L'ancora.** Apri la collezione, scrolla a metà e, dalla console, inserisci un `<div style="height:2000px">` come primo figlio di `.serie-collezione`. Il contenuto sotto i tuoi occhi si sposta? Ripeti dopo aver messo `overflow-anchor: none` su quel contenitore e spiega la differenza.
4. **Il "no" del database.** Metti una carta nella lista desideri, poi forza dalla console un evento `preferita-cambiata` per quella carta. Cosa vedi a schermo, e in quale ordine? Cosa vedresti se `impostaPreferita()` restituisse `void`?
5. **Attributi e cascata.** Nel cuore restano `stroke-width` e `stroke-linejoin` come attributi. Sono default ragionevoli o stato travestito? Scegli e giustifica con il criterio della sezione 5.
6. **Il costo che non si vede.** `aggiornaPreferita()` fa una `querySelectorAll('[data-preferita]')` su tutta la griglia per trovare una card. Con 800 carte a schermo, quanto costa davvero? Misuralo con `performance.now()` e di' se valga la pena tenere invece una mappa chiave → elemento — e cosa dovresti ricordarti di fare a ogni ridisegno.